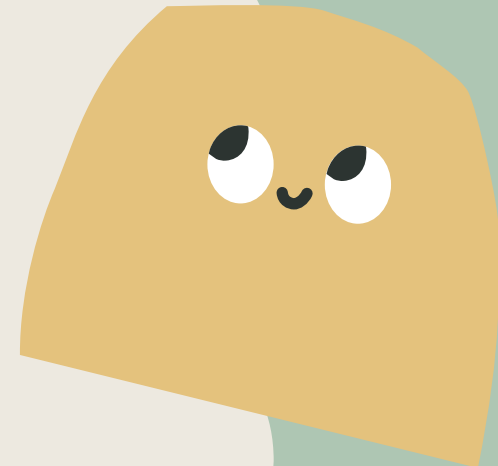
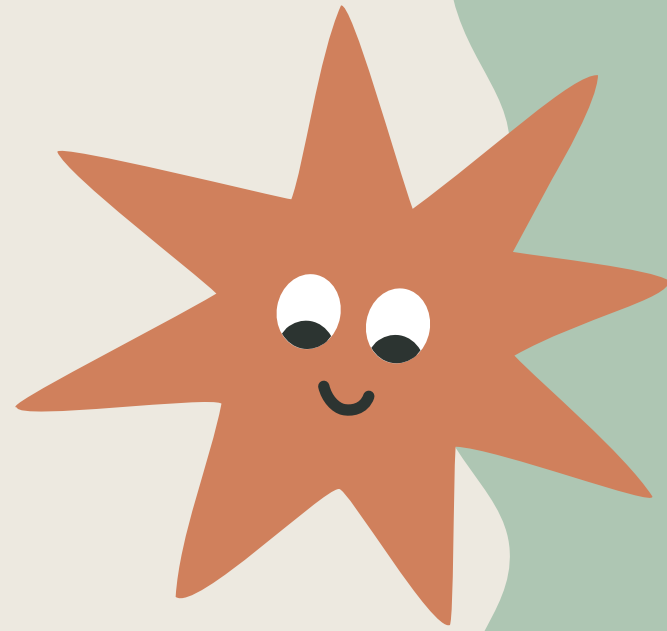


Doubly Linked List

Lecture 11



Learning Outcomes (Lecture 11)

- At the end of this lecture, students will be able to:
- Define a Doubly Linked List.
 - Explain the structure of a doubly linked-list node.
 - Explain the purpose of prev and next pointers.
 - Differentiate Doubly Linked Lists from Singly Linked Lists.
 - Traverse a Doubly Linked List in both directions.
 - Insert nodes at different positions.
 - Delete nodes from different positions.
 - Search and update nodes.
 - Analyze the time complexity of basic operations.
 - Identify practical applications of Doubly Linked Lists.

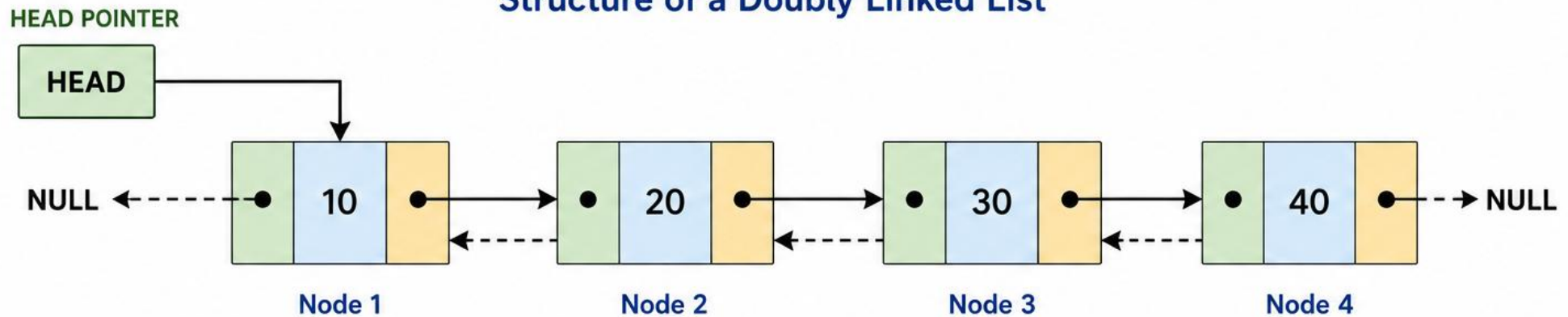
DOUBLY LINKED LIST

DEFINITION

A **doubly linked list** is a linear data structure in which each node contains three parts: a pointer to the **previous** node, the **data**, and a pointer to the **next** node. It allows traversal in both **forward** and **backward** directions.

STRUCTURE

Structure of a Doubly Linked List



Prev Pointer
(points to previous node)

Data
(value)

Next Pointer
(points to next node)

→ Link to Next Node
(Forward Direction)

← -- Link to Previous Node
(Backward Direction)

Practical Applications of Circular Linked List

➤ Browser History

- Each page can link to both the previous and next page

➤ Music / Video Playlist

- Song 1 ⇌ Song 2 ⇌ Song 3 ⇌ Song 4

➤ Undo / Redo in Software

- Action 1 ⇌ Action 2 ⇌ Action 3 ⇌ Action 4
- Undo → move backward, Redo → move forward

➤ Image / Photo Viewer

- Image 1 ⇌ Image 2 ⇌ Image 3 ⇌ Image 4
- Previous and Next buttons naturally map to prev and next

Basic Operations on Circular Linked List

➤ Following are the basic operations supported by a Linked List:

- **Traverse** – Print all the elements of linked list one by one
- **Insertion (4 Cases)** – Adds before an element, adds after an element, adds at start, add at end of linked list
- **Deletion (3 Cases)** – Delete by value, delete from start, delete at the end
- **Search** – Searches a node by the value
- **Update** – Updates a node's value

TRAVERSE DOUBLY LINKED LIST (FORWARD)

Forward traversal means visiting nodes from **HEAD** to the last node by following the **next** pointers.

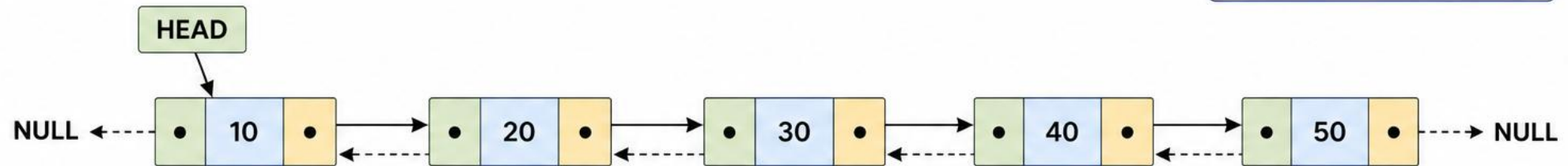
NODE STRUCTURE



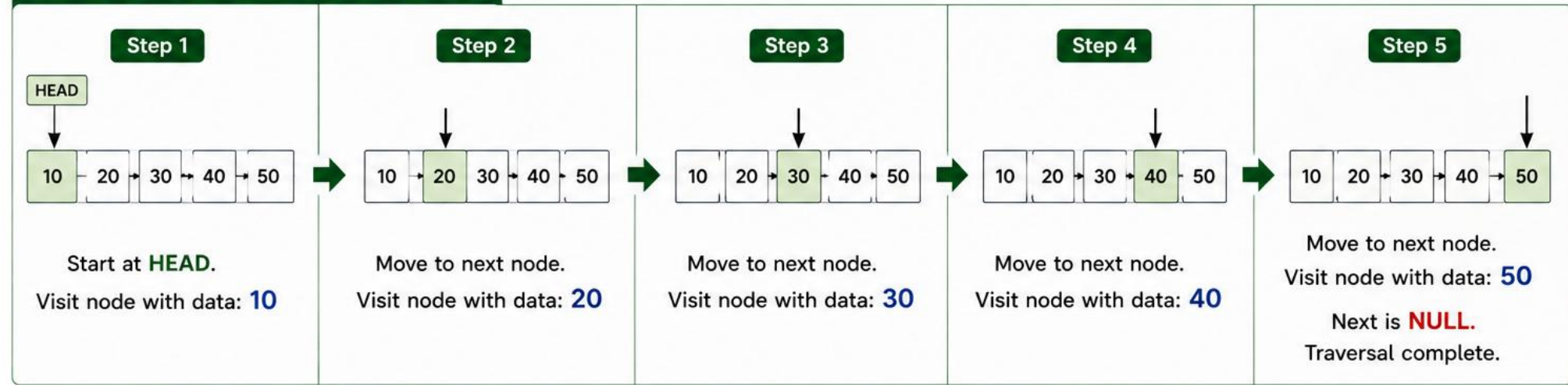
→ Link to Next Node

---> Link to Previous Node

DOUBLY LINKED LIST

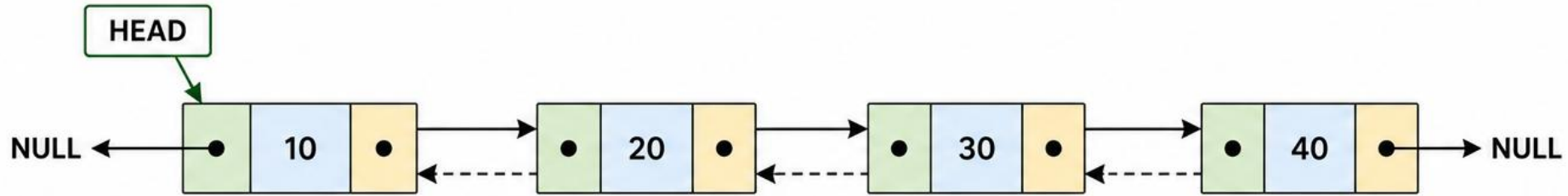


FORWARD TRAVERSAL (STEP BY STEP)



We start from **HEAD** and keep moving to the next node using **next** pointer until we reach **NULL**.

DOUBLY LINKED LIST – TRAVERSE



1) TRAVERSE FORWARD (From HEAD to NULL)

```
void traverseForward(Node* head) {
    Node* current = head;
    cout << "Forward Traversal: ";
    while (current != NULL) {
        cout << current->data << " ";
        current = current->next;
    }
    cout << endl;
}
```

2) TRAVERSE BACKWARD (From TAIL to NULL)

```
void traverseBackward(Node* tail) {
    Node* current = tail;
    cout << "Backward Traversal: ";
    while (current != NULL) {
        cout << current->data << " ";
        current = current->prev;
    }
    cout << endl;
}
```

EXPLANATION

- For forward traversal, start from **HEAD** and move using **next** pointer until **NULL**.
- For backward traversal, start from **TAIL** and move using **prev** pointer until **NULL**.

TIME COMPLEXITY (General)

$O(n)$

Where **n** is the number of nodes in the doubly linked list.



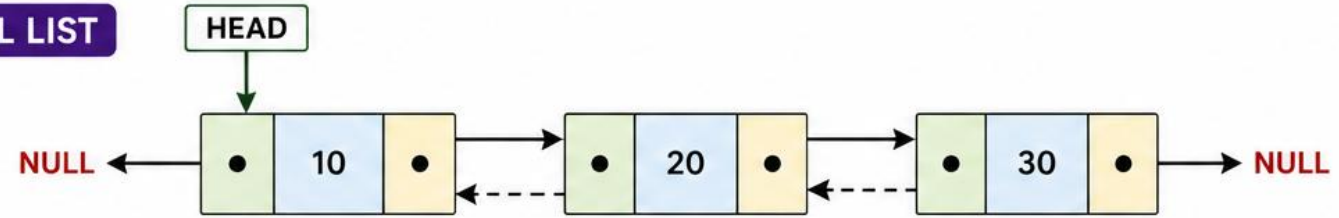
We may need to visit every node once in the list. Therefore, the time complexity is **$O(n)$** .

DOUBLY LINKED LIST – INSERTION AT START

OPERATION

Insert a new node at the beginning of the list.

INITIAL LIST

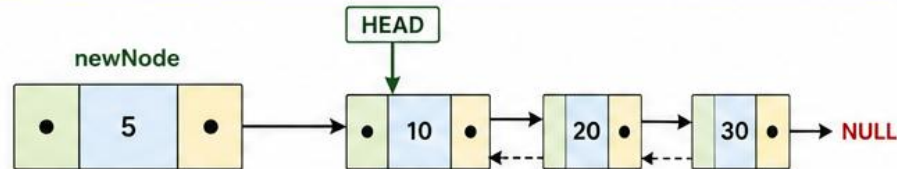


INSERT 5 AT START

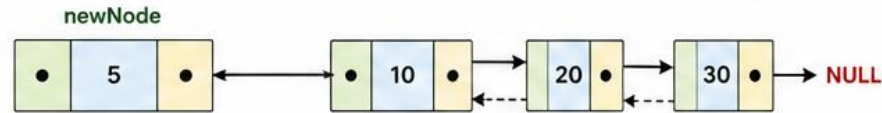
STEP 1 Create a new node with data = 5.



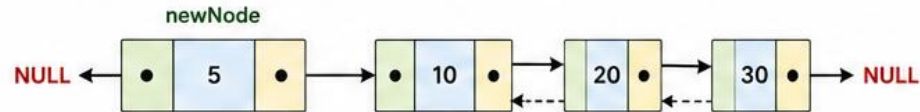
STEP 2 Make newNode's next point to the current head.



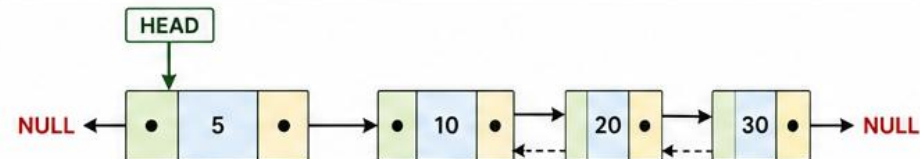
STEP 3 Make current head's prev point to newNode.



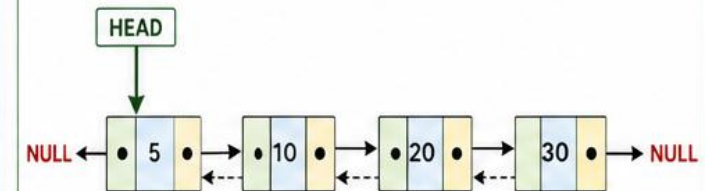
STEP 4 Make newNode's prev point to NULL.



STEP 5 Update HEAD to point to newNode.



FINAL LIST



SUMMARY OF STEPS

- 1 Create a new node.
- 2 $\text{newNode} \rightarrow \text{next} = \text{head}$
- 3 $\text{head} \rightarrow \text{prev} = \text{newNode}$
- 4 $\text{newNode} \rightarrow \text{prev} = \text{NULL}$
- 5 $\text{head} = \text{newNode}$

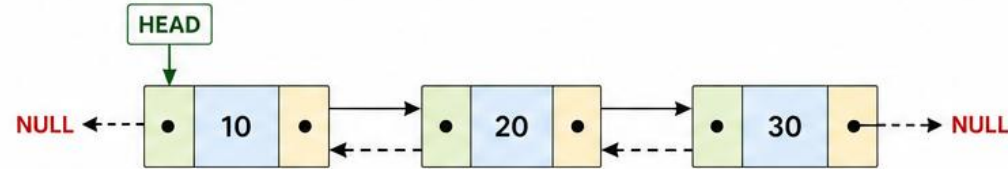
DOUBLY LINKED LIST – INSERTION AT END (NO TAIL POINTER)

OPERATION

Insert a new node at the end (after the current last node) in a Doubly Linked List.

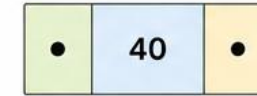
💡 We have only **HEAD** pointer.
No **TAIL** pointer is used.

INITIAL LIST



NEW NODE TO INSERT

Data = 40

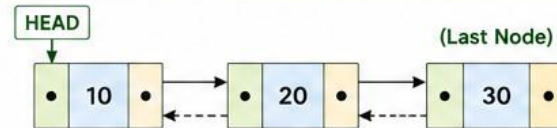


STEPS TO INSERT 40 AT END (WITHOUT TAIL POINTER)

STEP 1 Create a new node with data = 40.

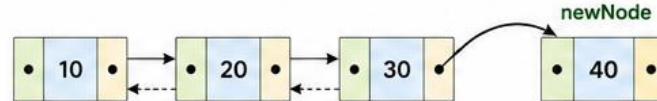


STEP 2 Start from **HEAD** and traverse the list to find the last node.

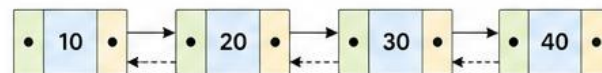


Travel using next pointer until next = NULL

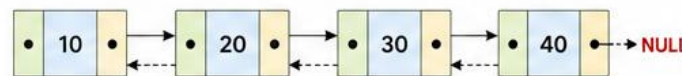
STEP 3 Make newNode's prev point to the last node.



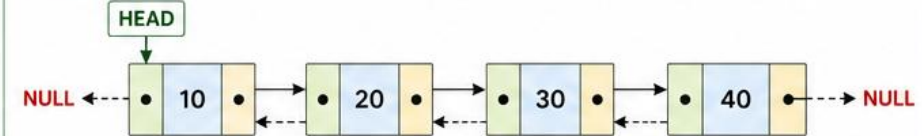
STEP 4 Make last node's next point to newNode.



STEP 5 Make newNode's next point to NULL.



FINAL LIST



SUMMARY OF STEPS

- 1 Create a new node with the given data.
- 2 Traverse from **HEAD** using next pointer until last node (whose next = NULL).
- 3 newNode->prev = lastNode
- 4 lastNode->next = newNode
- 5 newNode->next = NULL

TIME COMPLEXITY



$O(n)$

We must traverse the list to find the last node because we do not have a tail pointer.
Therefore, insertion at end takes $O(n)$ time.



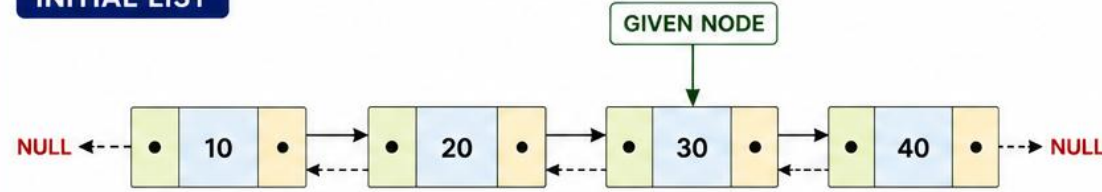
DOUBLY LINKED LIST – INSERT AFTER A GIVEN NODE

OPERATION

Insert a new node after a given (existing) node in a Doubly Linked List.

💡 A **pointer** (or reference) to the **given node** is required.

INITIAL LIST



NEW NODE TO INSERT

Data = 25



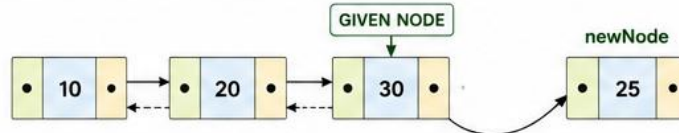
newNode

STEPS TO INSERT 25 AFTER THE GIVEN NODE (30)

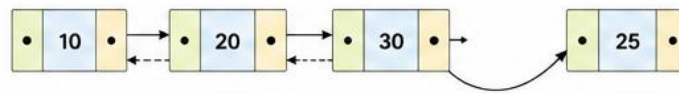
STEP 1 Create a new node with data = 25.



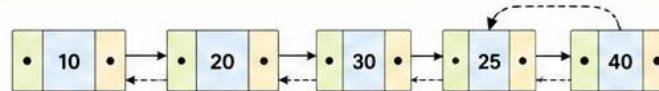
STEP 2 Make newNode's prev point to the given node (30).



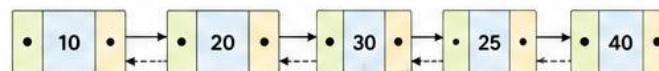
STEP 3 Make newNode's next point to the next node of the given node (40).



STEP 4 Make the next node (40) prev point to newNode.



STEP 5 Make the given node (30) next point to newNode.



FINAL LIST



SUMMARY OF POINTER UPDATES

- 1 Create a new node with the given data.
- 2 `newNode->prev = givenNode`
- 3 `newNode->next = givenNode->next`
- 4 `givenNode->next->prev = newNode`
- 5 `givenNode->next = newNode`

TIME COMPLEXITY



$O(n)$

To insert after a given node, we first need to locate the given node by traversing the list from the head (in the worst case). Therefore, the time complexity is $O(n)$.

📌 **Note:** If we already have a pointer (reference) to the given node, the insertion itself takes $O(1)$.



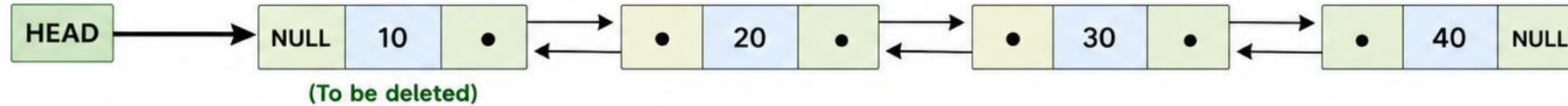
→ Link to Next Node

← --- Link to Previous Node

DOUBLY LINKED LIST: DELETION AT START

Deletion at start means removing the first node (head) from a doubly linked list.

INITIAL DOUBLY LINKED LIST



STEP-BY-STEP PROCESS

- Step 1** Check if the list is empty.
If **HEAD** is **NULL**, list is empty. Return.
- Step 2** Store the current head in a temporary pointer (temp).
- Step 3** Move **HEAD** to the next node.
HEAD = HEAD->next
- Step 4** Update the previous pointer of the new head to **NULL**.
HEAD->prev = NULL
- Step 5** Delete the old head node (temp).

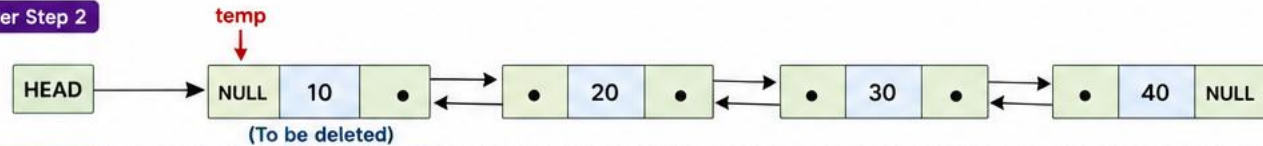
SUMMARY

The first node (10) is deleted.
The new head is now the node with data 20.

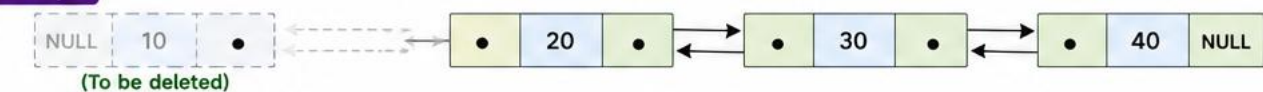
Before Deletion



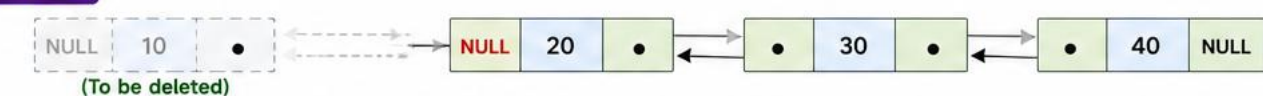
After Step 2



After Step 3



After Step 4



After Step 5



TIME COMPLEXITY



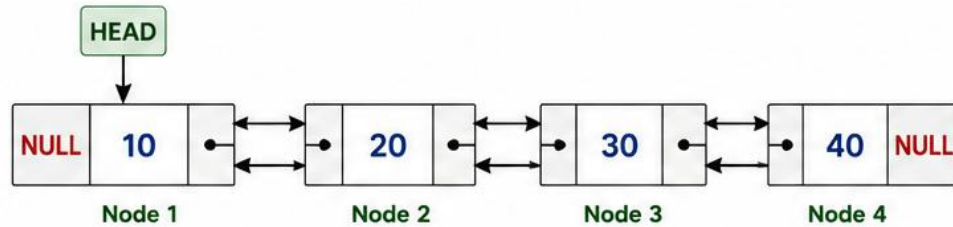
O(1)

Constant time operation.

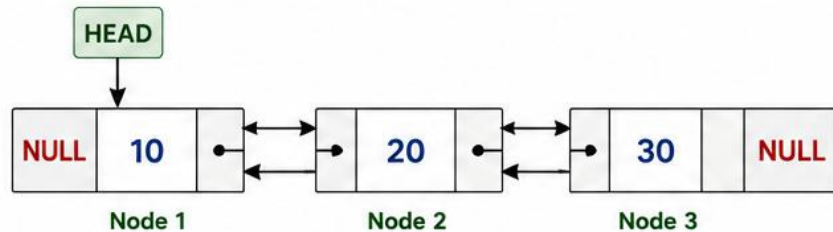
DOUBLY LINKED LIST: DELETION AT END (NO TAIL POINTER)

We will delete the last node **without using a tail pointer**.

INITIAL DOUBLY LINKED LIST



AFTER DELETION (Deleted 40)



WHAT HAPPENED?

- The last node (40) is deleted.
- The second last node (30) becomes the new last node.
- The new last node's next is set to NULL.

TIME COMPLEXITY



$O(n)$
(Linear Time)

Because without a tail pointer, we must traverse the list to find the second last node.

STEP-BY-STEP INSTRUCTIONS (Delete at End - Without Tail Pointer)

- 1 Check if the list is empty.
If HEAD = NULL, the list is empty. Return.
- 2 If the list has only one node:
 - Delete that node.
 - Set HEAD = NULL.
 - Return.
- 3 Create two pointers.
 - temp = HEAD
 - prev = NULL
(prev will always point to the node before temp)
- 4 Traverse the list to find the last node.
 - Move through the list until temp→next = NULL.
 - During the traversal, keep updating prev = temp and temp = temp→next.
 - After the loop ends, temp will be at the last node, and prev will be at the second last node.
- 5 Delete the last node (temp).
 - Delete temp.
- 6 Update the second last node.
 - Set prev→next = NULL.
(Now prev becomes the last node.)

IMPORTANT NOTE

Without a tail pointer, we must traverse the entire list to reach the last node, which makes the operation **$O(n)$** .